

Self-Balancing Robot

Outcome:

To design and implement a two-wheel self-balancing robot using an ESP32-C6 microcontroller, MPU6050 gyro/accelerometer sensor, and PID control algorithm.

Components:

Sno	Component Name	Quantity
1.	ESP32C6 Dev module	1
2.	USB to Type – C cable	1
4.	L298N Motor driver	1
5.	300 RPM BO motor with wheels	2
6.	3.7V 5000 mAh battery(18650 size)	2
7.	Two Battery holder with switch	1
8.	Jumper wires	As needed
9.	Acrylic sheets for Chassis	1

Concept:

A self-balancing robot works on the principle of an inverted pendulum, where the center of mass must be balanced dynamically by moving the wheels forward or backward.

The MPU6050 sensor combines a gyroscope and accelerometer to measure angular velocity and tilt angle. These values are processed using the Digital Motion Processor (DMP) inside the MPU6050 for more stable readings.

A PID controller (Proportional–Integral–Derivative) is used to calculate the corrective motor speed based on the tilt angle. The ESP32-C6 reads data from the MPU6050, computes the PID output, and drives the motors through the L298N motor driver to keep the robot upright.

Connections:

1. Hardware Setup:

- Connect the MPU6050 to ESP32-C6 using I2C (SDA → GPIO21, SCL → GPIO22).
- Connect the L298N motor driver to ESP32 pins: IN1 (6), IN2 (7), ENA (8), IN3 (4), IN4 (5), ENB (10).
- Connect two 300 RPM DC motors to the L298N outputs.
- Power the ESP32-C6 and L298N using a lithium-ion battery pack with a switch.

2. Software Setup:

- Install Arduino IDE.
- Install the following libraries via Library Manager:
 - I2Cdev- (Download [link](#))
 - PID_v1(Download [link](#))
 - MPU6050_6Axis_MotionApps20(Download [link](#))
- Click the Sketch option > Include Library > Add. Zip Library
- Install ESP32 board support in Arduino IDE.
- Select ESP32C6 Dev Module and choose the correct COM port.

3. Program Upload:

- Copy the given Arduino code into the IDE.
- Upload the sketch to ESP32-C6.

4. Testing & Operation:

- Place the robot upright on a flat surface.
- Power on the switch; the ESP32 initializes the MPU6050.
- The robot will attempt to balance itself using the PID algorithm.
- Adjust PID constants (Kp, Ki, Kd) in the code for smoother balancing.

Assembly Instructions

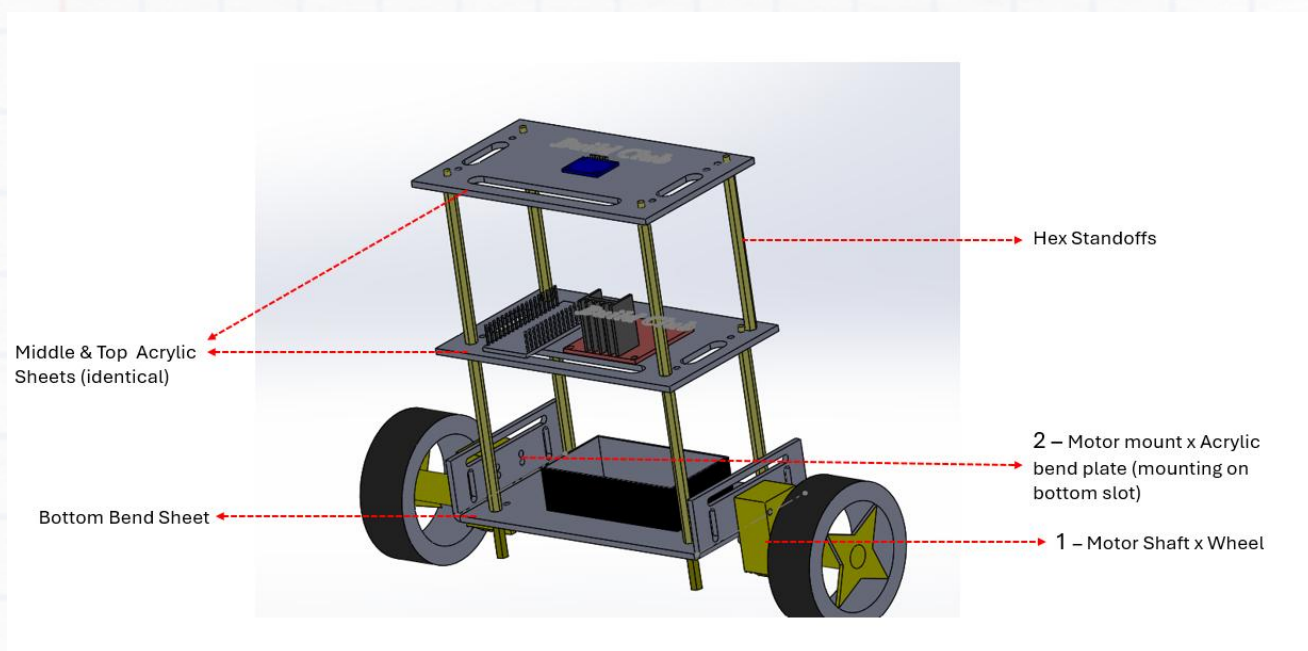
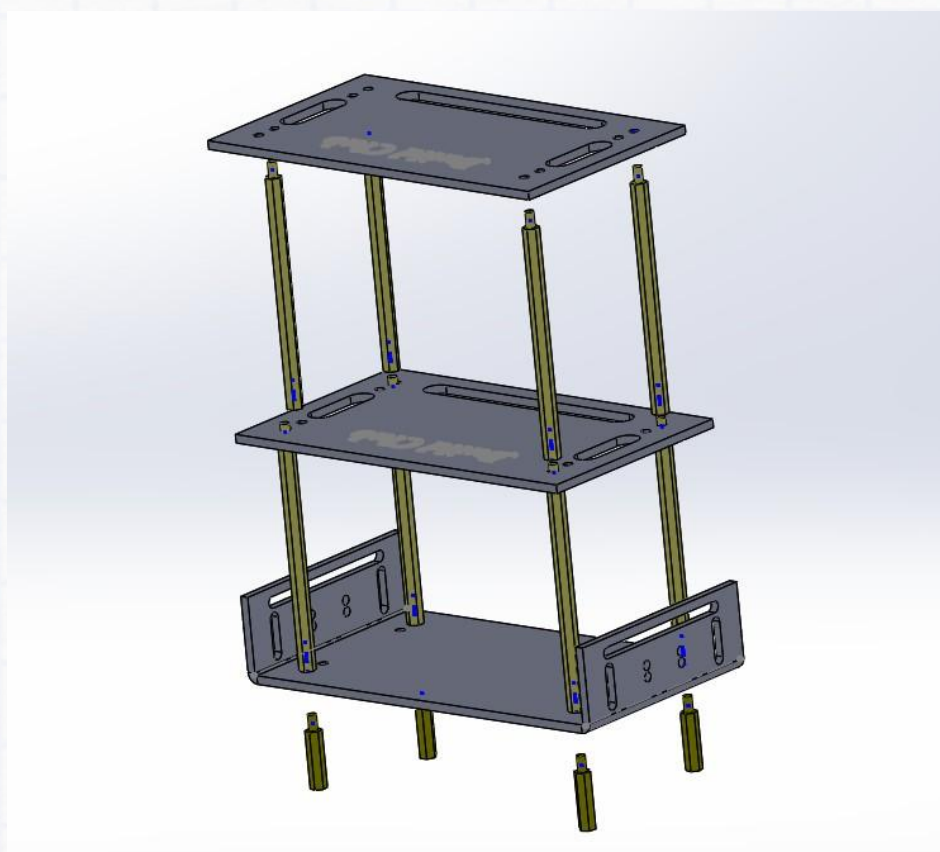
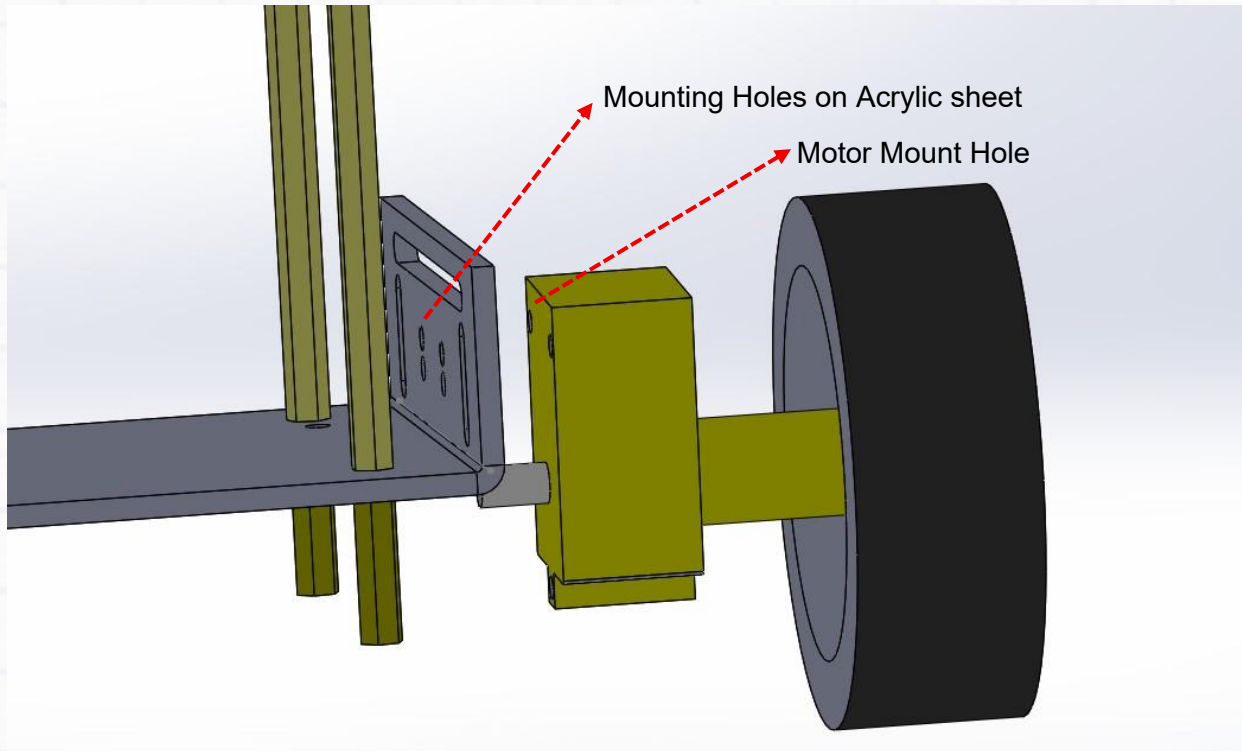


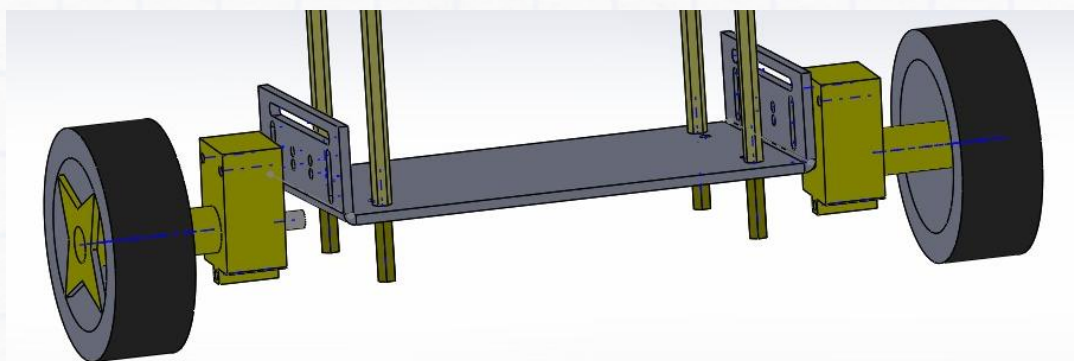
Fig: Assembled View



Step 1 – Mount the Acrylic Chassis as shown with the Hex studs & Nuts



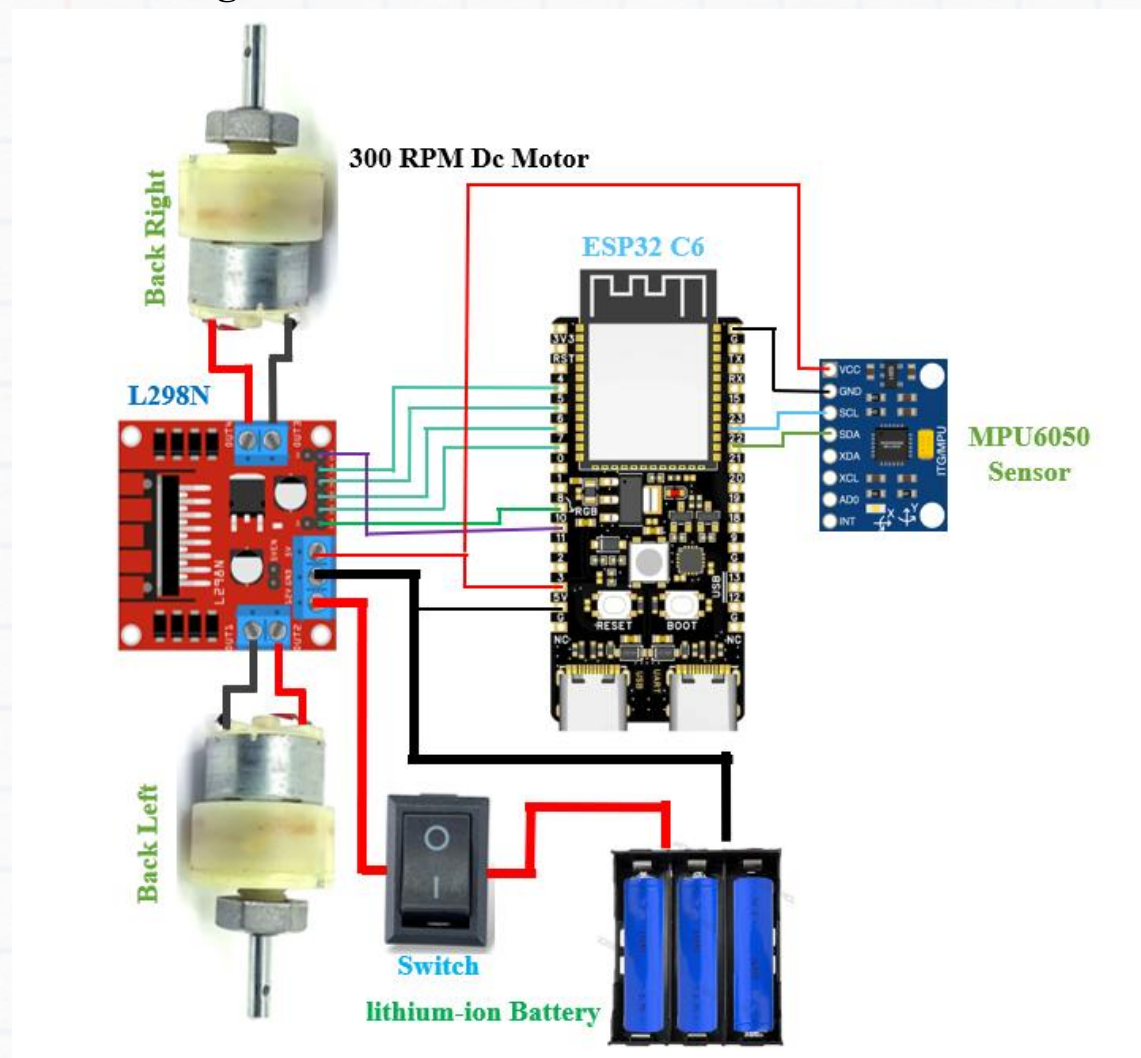
Step 2 – Mount the BO Motor to the bottom folded Acrylic sheet
(To be mounted in the appropriate slot with given bolts)



Step 3 – Mount the BO Motor on the other side of the bottom acrylic sheet

Place the other components on top of the acrylic sheets – Battery, MPU 6050, ESP 32 etc and wire them appropriately.

Circuit Diagram:



Software Code:

```
#include <Wire.h>
#include <I2Cdev.h>
#include <PID_v1.h>
#include "MPU6050_6Axis_MotionApps20.h"
#define SDA_PIN 21
#define SCL_PIN 22
// Motor A pins
#define IN1 6
#define IN2 7
#define ENA 8

// Motor B pins
```

```
#define IN3 4
#define IN4 5
#define ENB 10

MPU6050 mpu;

// PID variables
double input, output, setpoint = 0;
double Kp = 4.0; // robot oscillates fast.
double Ki = 10.0;
double Kd = 1.3;//dampen overshoot.

PID pid(&input, &output, &setpoint, Kp, Ki, Kd, DIRECT);

// MPU6050 DMP variables
bool dmpReady = false;
uint8_t devStatus;
uint16_t packetSize;
uint16_t fifoCount;
uint8_t fifoBuffer[64];
Quaternion q;
VectorFloat gravity;
float ypr[3];
float currentAngle;

void setup() {
  Serial.begin(115200);
  Wire.begin(SDA_PIN, SCL_PIN);
  // Set motor pins
  pinMode(IN1, OUTPUT); pinMode(IN2, OUTPUT); pinMode(ENA, OUTPUT);
  pinMode(IN3, OUTPUT); pinMode(IN4, OUTPUT); pinMode(ENB, OUTPUT);
  // Initialize MPU6050
  mpu.initialize();

  // Set MPU6050 offsets (tune these based on your calibration)
```

```
mpu.setXAccelOffset(-2500);
mpu.setYAccelOffset(-4000);
mpu.setZAccelOffset(1550);
mpu.setXGyroOffset(65);
mpu.setYGyroOffset(-30);
mpu.setZGyroOffset(20);

devStatus = mpu.dmpInitialize();

if (devStatus == 0) {
    mpu.setDMPEnabled(true);
    packetSize = mpu.dmpGetFIFOPacketSize();
    dmpReady = true;
    Serial.println("MPU6050 ready with DMP");
} else {
    Serial.print("DMP init failed (code ");
    Serial.print(devStatus);
    Serial.println(")");
}

pid.SetMode(AUTOMATIC);
pid.SetOutputLimits(-120, 120);
}

void loop() {
    if (!dmpReady) return;

    fifoCount = mpu.getFIFOCount();

    if (fifoCount == 1024) {
        mpu.resetFIFO();
        Serial.println("FIFO overflow!");
        return;
    }

    if (fifoCount >= packetSize) {
        mpu.getFIFOBytes(fifoBuffer, packetSize);
        mpu.dmpGetQuaternion(&q, fifoBuffer);
```

```

mpu.dmpGetGravity(&gravity, &q);
mpu.dmpGetYawPitchRoll(ypr, &q, &gravity);

currentAngle = ypr[1] * 180 / M_PI; // Pitch angle in degrees
input = currentAngle;

pid.Compute();
driveMotors(output);

Serial.print("Angle: ");
Serial.print(currentAngle);
Serial.print(" | Output: ");
Serial.println(output);
}
}

void driveMotors(double pidOut) {
    int motorSpeed = abs(pidOut);
    if (motorSpeed > 120) motorSpeed = 0;

    if (pidOut > 0) {
        // Move backward
        digitalWrite(IN1, LOW); digitalWrite(IN2, HIGH);
        digitalWrite(IN3, LOW); digitalWrite(IN4, HIGH);
    } else {
        // Move forward
        digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);
        digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW);
    }
    analogWrite(ENA, motorSpeed);
    analogWrite(ENB, motorSpeed);
}

```

Post Workshop Tasks (Club Activities):

1. Test the self-balancing robot on a ramp and fine tune its stability.

2. Test the stability of the robot by adding some weight on top of the chassis and balance it.